

Heidelberg University

Polyps Segmentation in Colonoscopy

Pranjal Sharma, Hanisha Kasaraneni

Advanced Practical Deep Learning for Biomedical Image Analysis

Summer Semester 2025

Supervisor: Prof. Dr. Karl Rohr

14.4.2025 – 15.7.2025

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 Goals	4
2 Fundamentals	5
2.1 EfficientNet	5
2.2 Swin Transformer	5
2.3 Multi-Dilation Convolution (MDC) Block	5
2.4 Multi-Feature Aggregation (MFA) Block	6
2.5 Decoder - CBAM Attention and Inception-Res Blocks	6
2.6 Loss formulation: Weighted Binary Cross Entropy and IoU	7
2.7 Metrics	8
2.8 Dataset	9
3 Previous Work	10
4 Methodology	12
4.1 Architecture	12
4.2 Loss and Training	13
4.3 Datasets and Evaluation	13
4.4 Engineering Adaptations	13
5 Experimental Results	15
5.1 Experiment Process	15
5.2 Results	16
5.3 Comparison	18
6 Conclusion and Future Work	19
7 Bibliography	20

Abstract

Colorectal Cancer (CRC) is among the most common cancers globally. CRCs develop from adenomatous polyps that are easy to detect and can be removed during colonoscopy. Early and accurate detection of polyps can improve the patient outcomes. However, polyps can be missed during colonoscopy because of smaller size and low-texture contrast. Therefore, minimizing the rate of missed detections is a critical goal. This report investigates the implementation of the SwinE-Net, a hybrid transformer-based architecture that uses the Swin Transformer to detect local contextual features, while the EfficientNet (CNN-based) captures global features of the polyps from the colonoscopy images.

According to the original SwinE-Net paper, the model achieves up to 0.938 mean Dice coefficient (mDice) and 0.892 mean Intersection over Union (mIoU) on benchmark datasets such as Kvasir-SEG and CVC-ClinicDB. It also generalizes well to unseen datasets like ColonDB, ETIS-Larib, and EndoScene, with reported results of up to 0.906 mDice and 0.842 mIoU.

In our experiments, we successfully reproduced the model and achieved comparable performance, with a peak mDice of 0.928 on Kvasir-SEG and 0.907 on ClinicDB.

Keywords: Colonoscopy, Polyp Segmentation, Swin Transformer, EfficientNet , Convolutional Neural Network.

1 Introduction

In this chapter, we discuss the motivation and the goal of our project.

1.1 Motivation

Colorectal cancer (CRC) is one of the leading causes of cancer-related death worldwide; early detection and removal of polyps during colonoscopy can reduce the incidence of CRC by up to 90 %. However, manual inspection has a miss rate of 20–30 % in routine practice, due to factors such as operator fatigue, poor visibility, and subtle appearance of the lesion as reported in [15]. Automating polyp segmentation, that is, drawing precise masks around suspected lesions, promises to (1) alert the endoscopist in real time, (2) reduce false negatives, and (3) standardize reporting between practitioners. Despite advances in Convolutional Neural Networks (CNNs) and vision transformers, most existing methods either lack global context or fail to preserve fine details, motivating a hybrid approach.

1.2 Goals

This report describes our work to reproduce and analyze the recent paper: “SwinE-Net: Hybrid deep-learning approach to novel polyp segmentation using convolutional neural network and Swin Transformer.” by Park et al. [8]. In the paper, the authors propose a new segmentation model, SwinE-Net, that combines an EfficientNet backbone (for fine details) with a Swin Transformer (for global context). They add multi-dilation convolutions, feature-fusion blocks, and attention-based decoding to produce highly accurate polyp masks [8]. In addition to implementing the original SwinE-Net architecture, we introduced minor architectural changes and evaluated their effects on performance. Our objective is to implement the SwinE-Net architecture from scratch following their descriptions of each module. In evaluation, we train and test on the public Kvasir-SEG and ClinicDB datasets to verify their reported metric values in Dice and IoU.

2 Fundamentals

In this section, the report discusses the key concepts and technologies used in this project.

2.1 EfficientNet

EfficientNet is a family of Convolutional neural networks (CNNs) designed to achieve higher accuracy with fewer parameters and less computational cost compared to previous models. Introduced by Tan and Le in 2019 [13], EfficientNet introduces a novel method called compound scaling, which uniformly scales a model's depth (number of layers), width (number of channels), and resolution (input image size) using a fixed set of scaling coefficients.

2.2 Swin Transformer

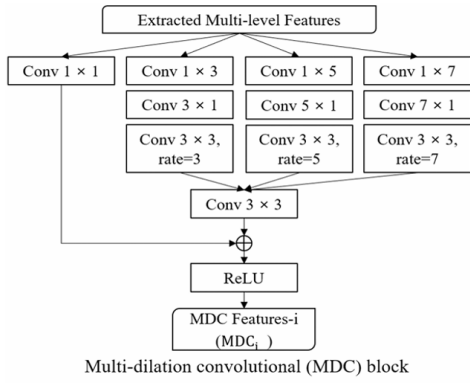
The Swin Transformer (Shifted Window Transformer) is a Transformer-based neural network architecture designed specifically for vision tasks. Unlike traditional Transformers that compute self-attention globally across the entire image, the Swin Transformer operates on smaller, non-overlapping windows. By gradually shifting these windows between layers, the Swin Transformer captures cross-window dependencies and builds a more comprehensive representation of the input [6].

2.3 Multi-Dilation Convolution (MDC) Block

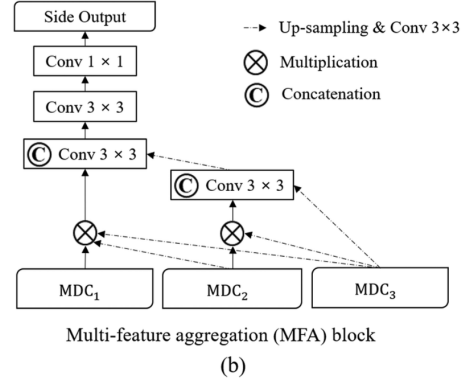
The Multi-Dilation Convolution (MDC) block is a module designed to capture multi-scale contextual information by applying Convolutional filters with different dilation rates in parallel. Dilation allows the network to expand the receptive field without

increasing the number of parameters or reducing resolution.

In the MDC block, multiple convolutional layers are applied with varying dilation rates. After applying the convolutions, the outputs from different dilation rates are either concatenated or aggregated to form a unified feature representation. This aggregated output is then passed on to the next layer or module in the network[8]. The architecture of the MDC block is shown in the Figure 2.1a.



(a) MDC block: Multiple dilated convolutions in parallel capture multi-scale context.



(b) MFA block: Fuses multi-scale features into a unified representation.

Figure 2.1: Architecture of the MDC and MFA blocks used in SwinE-Net.

2.4 Multi-Feature Aggregation (MFA) Block

The Multi-Feature Aggregation (MFA) block is a feature fusion module designed to combine features from different levels or branches of a neural network. The goal of the MFA block is to effectively integrate these multi-scale features to produce a richer and more informative representation. MFA takes low-level and high-level features as input, processes them through Convolutional or attention-based modules, and then aggregates them either using concatenation followed by a 1×1 convolution, or through an attention mechanism that adaptively weights the importance of each feature[8]. The architecture of MFA block is shown in the Figure 2.1b

2.5 Decoder - CBAM Attention and Inception-Res Blocks

To enhance feature representation and improve the focus on informative regions, the decoder integrates both CBAM (Convolutional Block Attention Module) and Inception-

Res blocks.

CBAM is a lightweight attention mechanism that sequentially applies channel and spatial attention to refine feature maps. [16] The module consists of channel attention that learns which feature channels are more important and spatial attention highlights significant spatial locations in the image.

Inception-Res blocks combine the parallel multi-scale convolutional paths of Inception modules with residual connections from ResNet [3]. This combination provides a strong balance between efficient multi-scale feature extraction, deeper representational capacity without degradation, and stable gradient flow during training.

2.6 Loss formulation: Weighted Binary Cross Entropy and IoU

To address class imbalance and improve segmentation performance, we employ a combined loss function consisting of weighted binary cross entropy (BCE) and Intersection over Union (IoU) loss.

- Weighted Binary Cross Entropy (BCE)¹: The weighted BCE loss is defined as:

$$\mathcal{L}_{\text{BCE}} = -[w_p \cdot y \cdot \log(\hat{y}) + w_n \cdot (1 - y) \cdot \log(1 - \hat{y})] \quad (1)$$

where y denotes the ground truth label, $\hat{y}$ is the predicted probability, w_p and w_n are the weights for the positive and negative classes, respectively.

- Weighted Intersection over Union (IoU) Loss²: The weighted IoU loss is calculated as:

$$\text{IoU}_{\text{weighted}} = \frac{\sum_i w_i \cdot (y_i \cdot \hat{y}_i)}{\sum_i w_i \cdot (y_i + \hat{y}_i - y_i \cdot \hat{y}_i)} \quad (2)$$

$$\mathcal{L}_{\text{IoU,weighted}} = 1 - \text{IoU}_{\text{weighted}} \quad (3)$$

- Total Loss: The total loss used for training is the sum of weighted BCE and weighted IoU loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE,weighted}} + \mathcal{L}_{\text{IoU,weighted}} \quad (4)$$

¹<https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html>

²https://github.com/qubvel/segmentation_models.pytorch

2.7 Metrics

This project used three metrics to evaluate SwinE-Net:

- **Mean Intersection over Union (mIoU):**

Mean Intersection over Union measures the overlap between the predicted segmentation and the ground truth [10]. It is computed as the average IoU over all classes:

$$\text{IoU}_i = \frac{|P_i \cap G_i|}{|P_i \cup G_i|} = \frac{TP_i}{TP_i + FP_i + FN_i}$$

$$\text{mIoU} = \frac{1}{N} \sum_{i=1}^N \text{IoU}_i$$

where P_i and G_i are the predicted and ground truth masks for class i , and N is the number of classes.

- **Mean Dice (mDice):**

The Dice coefficient measures the similarity between two samples [11]. The mean Dice is the average Dice score across all classes:

$$\text{Dice}_i = \frac{2|P_i \cap G_i|}{|P_i| + |G_i|} = \frac{2TP_i}{2TP_i + FP_i + FN_i}$$

$$\text{mDice} = \frac{1}{N} \sum_{i=1}^N \text{Dice}_i$$

Where: TP_i - true positives,

FP_i - false positives,

and FN_i - false negatives for class i .

- **Accuracy:**

Accuracy measures the proportion of correctly predicted pixels over the total number of pixels:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2.8 Dataset

This section describes the datasets used in this project. There are two datasets used in this project:

- **Kvasir-SEG**: A publicly available dataset comprising high-quality annotated images of gastrointestinal polyps. It contains diverse polyp images with pixel-level segmentation masks, making it valuable for training and evaluating segmentation models in colonoscopy image analysis [4].
- **ClinicDB**: A clinical dataset consisting of endoscopic images along with expert-annotated segmentation masks. It is widely used for benchmarking segmentation algorithms in real-world clinical scenarios, reflecting challenges such as varied lighting, occlusions, and image quality[15].

The dataset comparison is given in the table 2.1.

Table 2.1: Comparison of Kvasir-SEG and ClinicDB datasets

Feature	Kvasir-SEG	ClinicDB
Number of images	1000	Approx. 612
Image resolution	Varies from 332x487 to 1920x1072 pixels.	384×288 pixels
Annotation type	Pixel-level segmentation masks	Pixel-level segmentation masks
Content	Gastrointestinal polyp images	Endoscopy images from clinical practice
Data source	Public research dataset	Clinical collection
Image format	JPEG	TIF
Challenges	Variability in polyp size and shape	Lighting variation, occlusions, noise

3 Previous Work

In this section, we review the model proposed in the paper: “*SwinE-Net: Hybrid deep learning approach to novel polyp segmentation using convolutional neural network and Swin Transformer*” by Park and Lee [8].

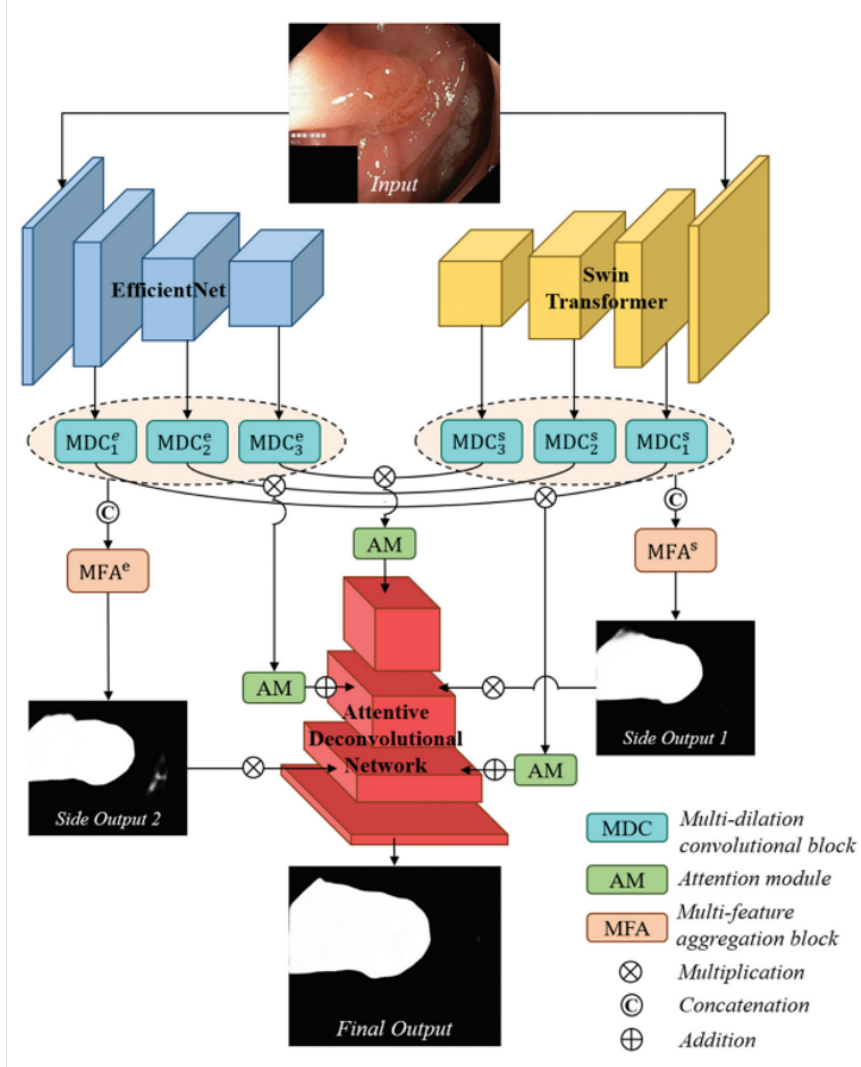


Figure 3.1: The model consists of a Swin Transformer encoder, MDC and MFA modules for context fusion, and CBAM-enhanced decoding.

Polyp segmentation has advanced significantly in recent years, primarily through the use of encoder–decoder architectures based on convolutional neural networks (CNN). Models like U-Net [11] and its extensions U-Net++ [20], ResUNet [19], and PraNet [2] have demonstrated strong performance in medical image segmentation tasks due to their ability to capture spatial features and reconstruct detailed segmentation masks. However, a common limitation of these CNN-based models is their difficulty in capturing long-range dependencies, which is critical when dealing with polyps that have irregular shapes, blurred boundaries, or low contrast [1].

To address this, more recent approaches have explored attention mechanisms and transformer-based encoders. Among them is SwinE-Net, proposed by Park and Lee [8], which takes a hybrid approach by combining the global modeling capacity of transformers with the spatial refinement capabilities of CNNs. The model uses a Swin Transformer as the encoder to extract multi-level contextual features efficiently through hierarchical, shifted-window self-attention.

As shown in Figure 3.1, SwinE-Net follows an encoder–decoder structure. The Swin Transformer encoder divides the input into patches and applies attention in local windows that shift across layers, allowing for efficient global context modeling. The decoder then processes the encoded features using three specialized modules: the Multi-Direction Context (MDC) module, which aggregates features from multiple spatial directions; the Multi-Feature Aggregation (MFA) module, which fuses feature maps from different resolutions; and the Convolutional Block Attention Module (CBAM), which applies both spatial and channel-wise attention to focus on important features.

The authors evaluated SwinE-Net on benchmark datasets such as Kvasir-SEG and CVC-ClinicDB. They used a combined loss function consisting of Binary Cross-Entropy (BCE) and Dice loss to balance pixel-wise accuracy with region-level overlap. The reported results showed notable improvements over previous CNN-based methods, particularly in terms of mean Dice and Intersection-over-Union (IoU) scores.

In summary, SwinE-Net is a well-designed hybrid model that effectively captures both global context and local detail. Its combination of transformer-based encoding and attention-enhanced convolutional decoding makes it particularly suited for polyp segmentation, where accurate boundary detection and context awareness are essential.

4 Methodology

Our model architecture, SwinE-Net, builds on the original work by Park and Lee [8] and is implemented entirely in PyTorch. While we follow their architectural blueprint, several engineering adaptations were introduced for better reproducibility and training efficiency.

4.1 Architecture

The encoder uses the Swin Transformer to extract hierarchical features using shifted window-based self-attention. This helps the model capture both local and global context with linear computational complexity.

The decoder includes:

- MDC (Multi-Direction Context): Approximated via multi-scale feature fusion to capture spatial features in multiple directions.
- MFA (Multi-Feature Aggregation): Integrated into upsampling blocks to combine feature maps of different resolutions.
- CBAM (Convolutional Block Attention Module): Applied within decoder blocks to enhance attention across spatial and channel dimensions.

Unlike the original implementation, we use `ConvTranspose2d` layers instead of bilinear interpolation followed by convolution. This combines upsampling and convolution into a single operation, reducing complexity.

4.2 Loss and Training

We use a weighted combination of Binary Cross-Entropy (BCE) and Dice Loss to balance pixel-wise classification with region overlap. Training was performed using:

- Adam optimizer with learning rate warm-up.
- Mixed precision training for faster convergence.
- Early stopping based on validation loss.
- Learning rate scheduler (`ReduceLROnPlateau` or Cosine Annealing in later experiments).

4.3 Datasets and Evaluation

We trained on Kvasir-SEG [4] and CVC-ClinicDB [15], both of which provide high-quality, pixel-level segmentation masks. To improve generalization, we applied a wide range of augmentations using the Albumentations¹ library, including flips, rotations, gamma correction, and advanced techniques such as CutMix² and MixUp³.

Evaluation was based on mean Dice coefficient (mDice), mean Intersection over Union (mIoU), precision, and recall. The experiments were run multiple times with different seeds, and average results were reported.

4.4 Engineering Adaptations

Our implementation introduces several modifications to enhance reproducibility:

- The MDC and MFA modules are not implemented as separate blocks but approximated through structured skip connections and decoder fusion logic.
- CBAM is implemented as a toggleable module and used selectively across decoder stages.

¹<https://albumentations.ai/>

²<https://github.com/clovaai/CutMix-PyTorch>

³<https://github.com/facebookresearch/mixup-cifar10>

These changes preserved the functional goals of SwinE-Net while simplifying the design and enabling more flexible experimentation.

Table 4.1 outlines the key architectural and implementation differences between our model and the original SwinE-Net proposed by Park and Lee [8]. These changes were made to improve training stability, simplify engineering, and leverage modern PyTorch-based tooling.

The changes listed in Table 4.1 reflect practical and engineering-driven decisions. For example, replacing bilinear + convolutional upsampling with `ConvTranspose2d` simplifies layer management and reduces manual operations. Instead of building dedicated modules for MDC and MFA from scratch, we designed a more flexible `UpsampleBlock` to encapsulate attention and fusion logic, reducing code duplication. We also leveraged modern data augmentation techniques such as CutMix and MixUp to boost generalization, especially important given the limited dataset sizes common in polyp segmentation.

Table 4.1: Key implementation differences between our model and the original SwinE-Net paper by Park and Lee

Aspect	SwinE-Net (Park and Lee)	Our Implementation
DataLoader	Custom data preprocessing (not described in detail)	Used <code>albumentations</code> for image augmentation, including flips, rotations, brightness adjustments, CutMix and MixUp for stronger generalization
Upsampling Type	Bilinear interpolation followed by convolution	Used <code>ConvTranspose2d</code> (deconvolution) layers, which combine upsampling and convolution in a single step
Skip Connections and Feature Fusion	Uses skip connections and MFA to combine multi-scale features; specific fusion sequence not detailed	Skip connections and MFAB used similarly, though fusion order and integration are customized within our blocks
Decoder Structure	Decoder composed of dedicated modules (MDC, MFA, CBAM)	Decoder uses reusable <code>UpsampleBlocks</code> that embed MFAB and optional CBAM, simplifying the overall design

5 Experimental Results

This chapter assesses the performance of the SwinE-Net model for the polyp segmentation. This report incorporates a series of seven experiments to find the best possible metrics. The objective is to evaluate whether the SwinE-Net model performs as good as it does in [8] with the new data. We emphasized mDice since it is less penalizing when a model misses thin edges or inaccurately segments fine polyp structures, unlike mIoU which drops more significantly. In this chapter we discuss the experiment process, results and comparison with the results in [8].

5.1 Experiment Process

All experiments were conducted using an NVIDIA A100 GPU with 40GB VRAM and 128GB system RAM via Google Colab¹. The software environment included Python 3.10, PyTorch 2.1, CUDA 12.1, and the following key libraries: OpenCV 4.8, albumentations 1.4, torchmetrics 1.7.3 and scikit-learn 1.4.

We trained and evaluated our models on the Kvasir-SEG and ClinicDB dataset, which consists of 1612 polyp images and corresponding masks. The dataset was split into 90% training and 10% testing.

Model selection was based on the best validation mDice score. Each experiment was repeated three times with different random seeds, and the average and standard deviation of the main metrics are reported. We have written the code in python which can be found in the Git Repository SwinE-Net². The code is saved as Polyp_Segmentation.ipynb.

The descriptions of the experiments are summarized in Table 5.1

table2.1

¹<https://colab.research.google.com>

²<https://github.com/djdubeyji/SwinE-Net.git>

Table 5.1: Experiment Description

Exp.	Description	Reason
1	Training dataset only	Test the working of the code
2	Testing and Training data split into 1:9	Test the model prediction quality
3	Changed the Loss function from BCEwithLogitsLoss to Weighted IoU and	Check which loss function fits the best
4	Added ClinicDB dataset	To train model on multiple dataset
5	Replaced Swin v1 transformer with Swin v2 transformer	Check which transformer performs the best
6	Advance augmentation (mixup/cutmix) + optimizer with weight decay + Reduce on Plateau Scheduler	Advance augmentation to improve input data, weight decay prevents overfitting and improve generalization, Scheduler helps to reduce learning rate when it starts overfitting.
6.1	Test data evaluated only for Kvasir-SEG dataset	Evaluate model's performance on Kvasir-SEG Dataset
6.2	Test data evaluated only for ClinicDB dataset	Evaluate model's performance on ClinicDB dataset
7	Replaced Reduce on Plateau Scheduler to Cosine Annealing	Check which Learning Rate Schedule performs the best

5.2 Results

In this section, we present and analyze the experimental results summarized in Tables 5.1 and 5.2.

Experiment 1 reports the metrics on the training data using BCE with Logits Loss over 6 epochs, primarily as a code sanity check. The high scores ($mDice = 0.949$, $mIoU = 0.927$) reflect fitting on the training set rather than generalization.

Experiment 2 evaluates the model on the Kvasir-SEG dataset using BCE with Logits Loss for 6 epochs. The dataset was split into training and testing in a 9:1 ratio. The model achieved $mDice = 0.894$ and $mIoU = 0.832$, establishing a baseline for further improvements.

Experiment 3 replaces the loss function with a combination of Weighted BCE and IoU loss, resulting in improved metrics ($mDice = 0.906$, $mIoU = 0.842$), while reducing the number of epochs to 5. This demonstrates the benefit of using a more task-specific loss.

Experiment 4 incorporates the ClinicDB dataset alongside Kvasir-SEG, with negligible

Table 5.2: Experiment Results

Exp	Dataset	Loss Function	Epochs	mDice	mIoU	Accuracy
1	Kvasir-SEG	BCE with Logit-sLoss	6	0.949	0.927	0.987
2	Kvasir-SEG	BCE with Logit-sLoss	6	0.894	0.832	0.969
3	Kvasir-SEG	Weighted BCE and IoU	5	0.906	0.842	0.971
4	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	5	0.90	0.841	0.975
5	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	15	0.875	0.802	0.969
6	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	15	0.899	0.865	0.978
6A	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	15	0.928	0.866	0.976
6B	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	15	0.907	0.830	0.982
7	Kvasir-SEG+ClinicDB	Weighted BCE and IoU	15	0.892	0.858	0.977

impact on the metrics ($\text{mDice} = 0.900$, $\text{mIoU} = 0.841$), indicating good cross-dataset generalization.

Experiment 5 substitutes the Swin V1 transformer backbone with Swin V2 to assess potential gains from the updated architecture. Contrary to expectations, the metrics decreased ($\text{mDice} = 0.875$, $\text{mIoU} = 0.802$), and the optimal number of epochs increased to 15.

Experiment 6 introduces advanced augmentations such as MixUp [18] and CutMix [17], employs the Adam optimizer with weight decay to mitigate overfitting, and applies the ReduceLROnPlateau scheduler [9] for adaptive learning rate adjustment. Metrics were reported on the combined dataset ($\text{mDice} = 0.899$, $\text{mIoU} = 0.865$), as well as separately for Kvasir-SEG ($\text{mDice} = 0.928$, $\text{mIoU} = 0.866$) and ClinicDB ($\text{mDice} = 0.907$, $\text{mIoU} = 0.830$). Notably, the Kvasir-SEG subset exhibited a significant performance increase compared to previous experiments.

Experiment 7 replaces the ReduceLROnPlateau scheduler with Cosine Annealing [7] for learning rate scheduling. This change resulted in a slight decrease in performance ($\text{mDice} = 0.892$, $\text{mIoU} = 0.858$).

Overall, these experiments demonstrate the progressive effect of loss function selection,

data augmentation, optimizer configuration, and learning rate scheduling on segmentation performance. The best results were obtained using weighted losses and advanced augmentation strategies, particularly on the Kvasir-SEG dataset.

5.3 Comparison

In this section, we compare the results reported in the original SwinE-Net paper [8] with those obtained in our experiments.

Table 5.3: Experiment results from the SwinE-Net paper [8]

Exp	Dataset	mDice	mIoU
1	Kvasir-SEG	0.920	0.870
2	ClinicDB	0.938	0.892

As shown in Tables 5.3 and 5.2, the original SwinE-Net achieved an mDice of 0.920 and mIoU of 0.870 on the Kvasir-SEG dataset, and an mDice of 0.938 and mIoU of 0.892 on the ClinicDB dataset. In contrast, our most promising results yielded an mDice of 0.928 and mIoU of 0.866 on the Kvasir-SEG dataset, and an mDice of 0.907 and mIoU of 0.830 on the ClinicDB dataset. These results indicate that our model slightly outperforms the original in terms of mDice on the Kvasir-SEG dataset, while performance on the ClinicDB dataset, although competitive, does not reach the same level as reported in the original paper.

6 Conclusion and Future Work

This project set out to reproduce and analyze the SwinE-Net model for polyp segmentation. We re-implemented the architecture from scratch, introducing engineering adaptations to improve modularity, training speed, and reproducibility.

The experiments show that our version performs competitively, achieving a higher mDice on the Kvasir-SEG dataset than reported in the original paper. However, performance on ClinicDB was slightly lower, likely due to dataset-specific variability and the challenges of generalization.

Our experiments revealed that data augmentation, customized loss functions, and improved upsampling strategies had a more significant impact on performance than adopting newer architectures such as Swin V2. The original SwinE-Net design remains strong, and with a few smart tweaks, can still hold its ground.

Future directions could include adapting the model for real-time inference, integrating clinical priors, or exploring few-shot generalization on rare polyp types. Overall, this work confirms that SwinE-Net is a valuable architecture and more importantly, that thoughtful engineering still matters.

7 Bibliography

- [1] J. Chen, Y. Lu, Q. Yu, X. Luo, E. Adeli, Y. Wang, L. Lu, A. Yuille, & Y. Zhou, “Transunet: Transformers make strong encoders for medical image segmentation,” *arXiv preprint arXiv:2102.04306*, 2021.
- [2] D.-P. Fan, G.-P. Ji, T. Zhou, G. Chen, H. Fu, & J. Shen, “Pranet: Structure-aware network for polyp segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6687–6696.
- [3] K. He, X. Zhang, S. Ren, & J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [4] D. Jha, P. H. Smedsrud, M. A. Riegler, P. Halvorsen, T. de Lange, D. Johansen, & H. D. Johansen, “Kvasir-seg: A segmented polyp dataset,” in *MultiMedia Modeling (MMM 2020)*, Y. M. Ro, A. Badii, S. Cagnoni, A. Dziech, M. Aiello, M. Morisio, & S. F. Ong, Eds., ser. Lecture Notes in Computer Science, vol. 11962, Springer, Cham, 2020, pp. 451–462. DOI: [10.1007/978-3-030-37734-2_37](https://doi.org/10.1007/978-3-030-37734-2_37).
- [6] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, & B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022. DOI: [10.1109/ICCV48922.2021.00988](https://doi.org/10.1109/ICCV48922.2021.00988).
- [7] I. Loshchilov & F. Hutter, “Sgdr: Stochastic gradient descent with warm restarts,” *International Conference on Learning Representations (ICLR)*, 2017.
- [8] K.-B. Park & J. Y. Lee, “Swine-net: Hybrid deep learning approach to novel polyp segmentation using convolutional neural network and swin transformer,” *Journal of Computational Design and Engineering*, vol. 9, no. 2, pp. 616–632, April 2022. DOI: [10.1093/jcde/qwac018](https://doi.org/10.1093/jcde/qwac018).
- [9] PyTorch, *Learning rate schedulers in pytorch*, <https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate>, Accessed: 2025-07-14.
- [10] M. A. I. Rahman & Y. Wang, “Optimizing intersection-over-union in deep neural networks for image segmentation,” in *International Symposium on Visual Computing (ISVC)*, 2016, pp. 234–244. DOI: [10.1007/978-3-319-50835-1_22](https://doi.org/10.1007/978-3-319-50835-1_22).

- [11] O. Ronneberger, P. Fischer, & T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. Lecture Notes in Computer Science, vol. 9351, Springer, Cham, 2015, pp. 234–241. DOI: [10.1007/978-3-319-24574-4_28](https://doi.org/10.1007/978-3-319-24574-4_28).
- [13] M. Tan & Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” *ArXiv*, vol. abs/1905.11946, 2019.
- [15] D. Vazquez, J. Bernal, F. J. Sanchez, G. Fernandez-Esparrach, A. M. Lopez, A. Romero, M. Drozdal, & A. Courville, “A benchmark for endoluminal scene segmentation of colonoscopy images,” *Journal of Healthcare Engineering*, vol. 2017, p. 4037190, 2017. DOI: [10.1155/2017/4037190](https://doi.org/10.1155/2017/4037190).
- [16] S. Woo, J. Park, J.-Y. Lee, & I. S. Kweon, “Cbam: Convolutional block attention module,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Springer, 2018, pp. 3–19. DOI: [10.1007/978-3-030-01234-2_1](https://doi.org/10.1007/978-3-030-01234-2_1).
- [17] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, & Y. Yoo, “Cutmix: Regularization strategy to train strong classifiers with localizable features,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6023–6032. DOI: [10.1109/ICCV.2019.00612](https://doi.org/10.1109/ICCV.2019.00612).
- [18] H. Zhang, M. Cisse, Y. N. Dauphin, & D. Lopez-Paz, “Mixup: Beyond empirical risk minimization,” in *International Conference on Learning Representations (ICLR)*, OpenReview ID: BJrFC6T45, 2018.
- [19] Z. Zhang, Q. Liu, & Y. Wang, “Road extraction by deep residual u-net,” *IEEE Geoscience and Remote Sensing Letters*, vol. 15, no. 5, pp. 749–753, 2018.
- [20] Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, & J. Liang, “Unet++: A nested u-net architecture for medical image segmentation,” in *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, Springer, 2018, pp. 3–11.

Project Timetable Comparision

Table 1: Timeline and Task Allocation

Week	Pranjal	Hanisha
Week 0	PPT-1	PPT-1
Week 1 – Kickoff	Literature Review	Understand SwinE-Net architecture
Week 2 – Dataset Collection and Preprocessing	Gather Dataset	Dataset Cleaning
Week 3 – CNN and Transformer Model Setup	Implement E-Net based CNN feature extractor	Implement Swin Transformer
Week 4 – Architecture Development	Integrate E-Net and Swin Transformer	Implement multidilation convolution block and multifeatured aggregation block
Week 5 – Decoder and Attention Mechanisms	Implement attentive deconvolution network	Integrate and test CBAM (Convolutional Block Attention Module)
Week 6 – Loss Function and Training Setup	Implement the loss function, Prepare PPT-2	Set up the training pipeline, Prepare PPT-2
Week 7 – Training the Model	PPT-2, Train and adjust the model	Conduct initial testing and evaluate early performance
Week 8 – Evaluation and Fine Tuning	Evaluate model performance on unseen datasets (Maybe not)	Perform fine Tuning
Week 9 – Verification and Report Creation	Verify results, clean git, verify documentation	Finalize report
Week 10	Project Presentation-3 and Report Submission	

***Predicted and actual timelines are the same.**